

Multithreading Enhancement

15 March 2026 10:10

ThreadGroup: Based on functionality we can group threads into single unit which is nothing but thread group. ThreadGroup contains a group of thread. In addition to threads, ThreadGroup also contains sub-thread groups.

The main advantage of maintaining threads in the form of thread group is we can perform common operations very easily.

In Java multi-threading, a ThreadGroup is a mechanism used to organize multiple threads into a simple group so that they can be managed collectively.

Instead of controlling each thread individually, a thread group allows operations to be performed on all thread in a group simultaneously. Such as interrupting, suspending, resuming, or setting priority.

Every thread in Java belongs to some group. Main thread belongs to main group. Every thread group in Java is the child group of system group either directly or indirectly. Hence system group act as root for all thread groups in Java.

System group contains several system level threads like: Finalizer, reference threads, signal Dispatcher, attach listener, etc.

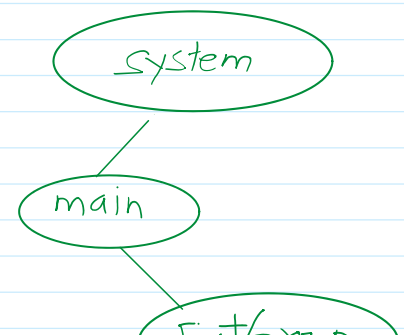
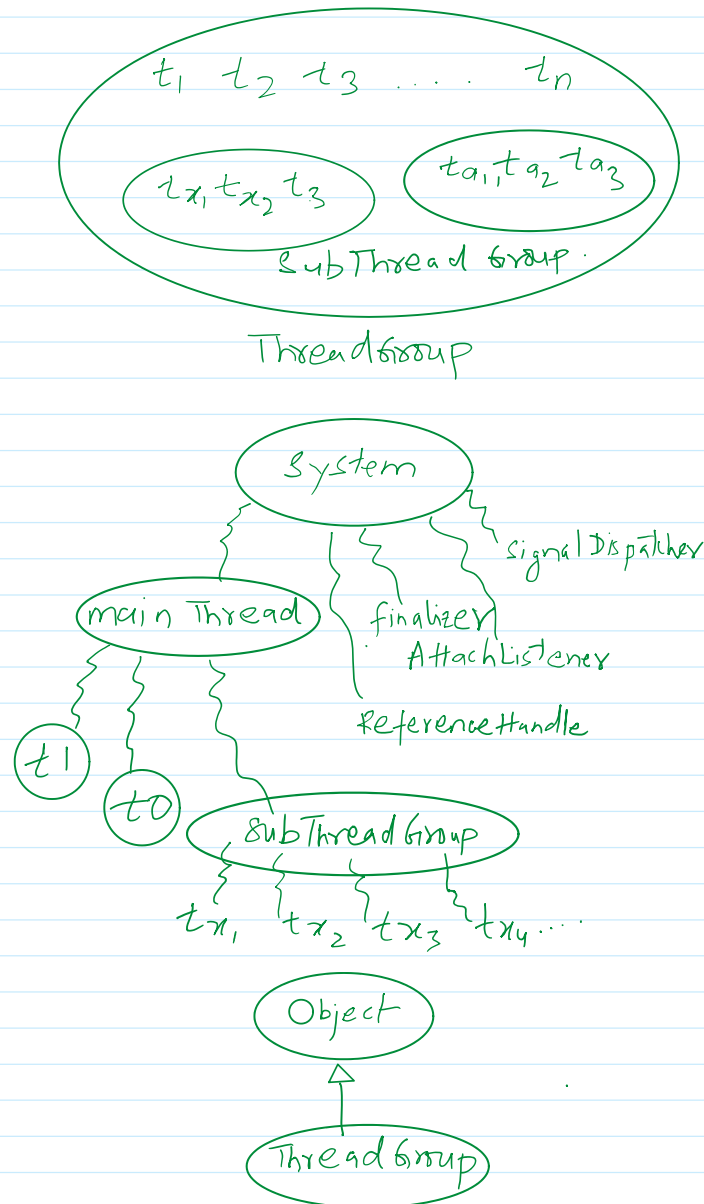
ThreadGroup is a Java class present in java.lang package and it is direct child class of Object class.

Constructors of ThreadGroup class:

`ThreadGroup g = new ThreadGroup(String gname); =>` Creates a new thread. Object with the specified group name. The parent of this new group is the ThreadGroup of currently executing thread.

`ThreadGroup g = new ThreadGroup(ThreadGroup pg, String gname); =>` Creates a new thread group with a specified group name. The parent of this new thread group is specified parent group object in the parameter.

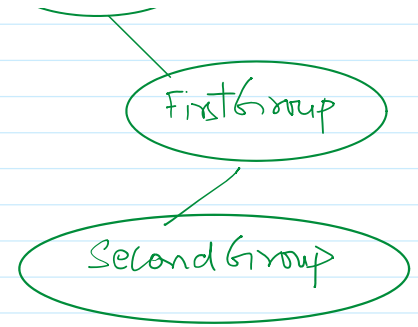
```
package MyThreads.MultithreadingEnhancement;
public class ThreadGroup1 {
    public static void main(String[] args) {
        ThreadGroup g1 = new ThreadGroup("First group");
        System.out.println(g1.getParent().getName()); //main
        ThreadGroup g2 = new ThreadGroup(g1, "Second group");
        System.out.println(g2.getParent().getName());
    }
}
```



```

ThreadGroup g2 = new ThreadGroup(g1, "Second Group");
System.out.println(g2.getParent().getName());
}
}

```



Output:

```

main
First group

```

Important methods of ThreadGroup class.

int	activeCount()	✓✓	Returns an estimate of the number of live platform threads in this thread group and its subgroups.
int	activeGroupCount()	✓✓	Returns an estimate of the number of groups in this thread group and its subgroups.
final void	checkAccess()		Deprecated, for removal: This API element is subject to removal in a future version. This method originally determined if the currently running thread had permission to modify this thread group.
final void	destroy()		Deprecated, for removal: This API element is subject to removal in a future version. This method was originally specified to destroy an empty thread group.
int	enumerate(Thread[] list)		Copies into the specified array every live platform thread in this thread group and its subgroups.
int	enumerate(Thread[] list, boolean recurse)		Copies into the specified array every live platform thread in this thread group.
int	enumerate(ThreadGroup[] list)		Copies into the specified array references to every subgroup in this thread group and its subgroups.
int	enumerate(ThreadGroup[] list, boolean recurse)		Copies into the specified array references to every subgroup in this thread group.
final int	getMaxPriority()	✓✓	Returns the maximum priority of this thread group.
final String	getName()	✓✓	Returns the name of this thread group.

final ThreadGroup	getParent()	✓✓	Returns the parent of this thread group.
final void	interrupt()	✓✓	Interrupts all <i>live</i> platform threads in this thread group and its subgroups.
final boolean	isDaemon()		Deprecated, for removal: This API element is subject to removal in a future version. This method originally indicated if the thread group is a <i>daemon thread group</i> that is automatically destroyed when its last thread terminates.
boolean	isDestroyed()		Deprecated, for removal: This API element is subject to removal in a future version. This method originally indicated if the thread group is destroyed.
void	list()	✓✓	Prints information about this thread group to the standard output.
final boolean	parentOf(ThreadGroup g)	✓✓	Tests if this thread group is either the thread group argument or one of its ancestor thread groups.
final void	setDaemon(boolean daemon)		Deprecated, for removal: This API element is subject to removal in a future version. This method originally configured whether the thread group is a <i>daemon thread group</i> that is automatically destroyed when its last thread terminates.
final void	setMaxPriority(int pri)	✓✓	Sets the maximum priority of the group.
String	toString()		Returns a string representation of this Thread group.
void	uncaughtException(Thread t, Throwable e)		Called by the Java Virtual Machine when a thread in this thread group stops because of an uncaught exception, and the thread does not have a specific Thread.UncaughtExceptionHandler

```
package MyThreads.MultithreadingEnhancement;
public class ThreadGroup2Demo {
    public static void main(String[] args) {
        ThreadGroup g1 = new ThreadGroup("First group");
        Thread t1 = new Thread(g1, "First thread");
        Thread t2 = new Thread(g1, "Second Thread");
        Thread t3 = new Thread(g1, "Third Thread");
        g1.setMaxPriority(3);
        System.out.println("Priority of thread t1: "+t1.getPriority()); //5
        System.out.println("Priority of thread t2: "+t2.getPriority()); //5
        System.out.println("Priority of thread t3: "+t3.getPriority()); //5
        System.out.println("Priority of group g1: " + g1.getMaxPriority()); //3
    }
}
```

Output:

```
Priority of thread t1: 5
Priority of thread t2: 5
Priority of thread t3: 5
Priority of group g1: 3
```

Threads in that have already higher priority won't be affected, but for newly added thread this Max priority is available.

```
package MyThreads.MultithreadingEnhancement;
public class ThreadGroup2Demo {
    public static void main(String[] args) {
        ThreadGroup g1 = new ThreadGroup("First group");
        Thread t1 = new Thread(g1, "First thread");
```

```

        Thread t2 = new Thread(g1, "Second Thread");
        Thread t3 = new Thread(g1, "Third Thread");
        g1.setMaxPriority(3);
        System.out.println("Priority of thread t1: "+t1.getPriority()); //5
        System.out.println("Priority of thread t2: "+t2.getPriority()); //5
        System.out.println("Priority of thread t3: "+t3.getPriority()); //5
        System.out.println("Priority of group g1: " + g1.getMaxPriority()); //3
    }
}

```

Output:

```

Priority of thread t1: 5
Priority of thread t2: 5
Priority of thread t3: 5
Priority of thread t4: 3
Priority of group g1: 3

```

Another example of ThreadGroup:

```

package MyThreads.MultithreadingEnhancement;
public class MyThread extends Thread {
    public MyThread(ThreadGroup g,String name){
        super(g, name);
    }
    @Override
    public void run() {
        System.out.println("Child Thread");
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

package MyThreads.MultithreadingEnhancement;
public class ThreadGroup3Demo {
    public static void main(String[] args) {
        ThreadGroup pg = new ThreadGroup("Parent Group");
        ThreadGroup cg = new ThreadGroup(pg, "Child Group");
        MyThread t1 = new MyThread(pg, "Child Thread-1");
        MyThread t2 = new MyThread(pg, "Child Thread-2");
        t1.start();
        t2.start();
        System.out.println(pg.activeCount());
        System.out.println(pg.activeGroupCount());
        pg.list();
    }
}

```

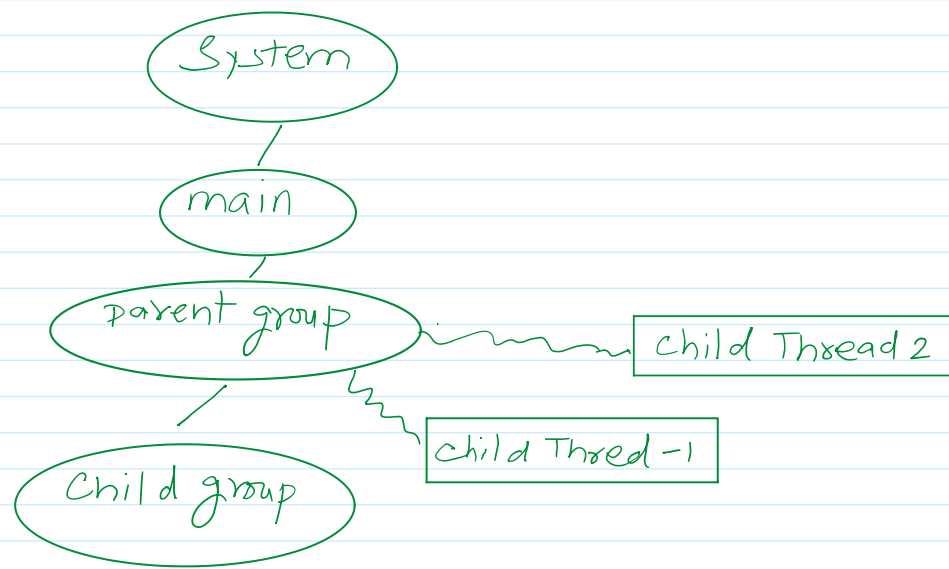
Output:

```

2
Child Thread
Child Thread
1
java.lang.ThreadGroup[name=Parent Group,maxpri=10]
  Thread[#21,Child Thread-1,5,Parent Group]

```

Thread[#22,Child Thread-2,5,Parent Group]
java.lang.ThreadGroup[name=Child Group,maxpri=10]



WAP to display all the active thread name belongs to system group and its child groups.

In Java, threads are organized into ThreadGroups.

The System Thread Group is the root (top-level) group.

- It contains

 - ⇒ Main thread group.

 - ⇒ Other child group.

- We can traverse all threads using:

 - ⇒ ThreadGroup

 - ⇒ activeCount()

 - ⇒ enumerate()

Approach:

1. Get the root thread group (System group)
2. Get total active threads.
3. Store them in an array
4. Display thread names

```

package MyThreads.MultithreadingEnhancement;

public class SystemThreads {
    public static void main(String[] args) {
        // 1. Get current thread group
        ThreadGroup currentGroup = Thread.currentThread().getThreadGroup();
        // 2. Move to root(System Thread Group)
        ThreadGroup rootGroup = currentGroup;
        while (rootGroup.getParent() != null) {
            rootGroup = rootGroup.getParent();
        }
        // 3. Get active thread count
        int count = rootGroup.activeCount();
        // 4. Create array to hold threads
        Thread[] threads = new Thread[count];
        // 5. Copy active threads into array
        rootGroup.enumerate(threads, true);
        // 6. Display thread names
        System.out.println("Active threads in System Thread Group:");
        for(Thread t: threads){
            if(t!=null){
                System.out.println(t.getName());
            }
        }
    }
}

// output
// main
// Reference handler
// Finalizer
// Signal Dispatcher
// Notification Thread
// Common-Cleaner
// (Output may vary depending on JVM and system)

```